

Reviewer Pack — Young Tableaux Count Lower Bound on Disjointness Communicati...

Entry 221ada155c09 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 00:57:01 UTC

Young Tableaux Count Lower Bound on Disjointness Communication Complexity

Entry ID: 221ada155c09 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 00:57:01 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Communication Complexity (Disjointness Problem)

Statement:

For a disjointness problem with n variables, the communication complexity is at least $\Omega(\sqrt{\text{number of standard Young tableaux of shape } (n, n)})$, where the number is calculated via the hook-length formula.

Rationale (proposer’s reasoning):

The number of standard Young tableaux of shape (n, n) corresponds to the dimension of the symmetric group representation, which might influence the rank of the communication matrix, leading to a lower bound on communication complexity.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 17560a4217b9b962

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def hook_length_formula(n):
    table = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            table[i][j] = (n - i) + (n - j) - 1
    det = determinant(table)
    return math.factorial(n**2) // det

def determinant(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    elif n == 2:
        return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
    else:
        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += (-1) ** j * matrix[0][j] * determinant(submatrix)
        return det

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        tableaux_count = hook_length_formula(n)
        lower_bound = math.log(tableaux_count) / math.log(2)
        # Estimate communication complexity using log-rank conjecture
        estimated_complexity = math.sqrt(lower_bound)
        actual_complexity = random.uniform(0, 2 * estimated_complexity) # Simulate actual complexity

        if actual_complexity < estimated_complexity:
```

```

        conjecture_holds = False
        counterexample = f"n={n}: Estimated {estimated_complexity}, Actual {actual_complexity}"

        total_metric_value += actual_complexity
        instances_tested += len(n_values)

    return {
        "metric_name": "communication_complexity",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2*i + 1 for i in range(5, 8)] # First 30 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cfad8ba1.py", line 76, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cfad8ba1.py", line 48, in run_trial
    tableaux_count = hook_length_formula(n)
                     ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cfad8ba1.py", line 24, in hook_length_formula
    return math.factorial(n**2) // det
           ~~~~~^~~~~~
ZeroDivisionError: integer division or modulo by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing evaluation of the conjecture's validity | next: Fix the hook-length formula implementation to handle division by zero and re-run tests

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	87866
2	preregistration	ollama_remote	qwen3:8b	0	0	23882
3	novelty	ollama_remote	qwen3:8b	0	0	20464
4	novelty	ollama_remote	qwen3:8b	0	0	15281
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12119
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8764
7	judge	ollama_remote	qwen3:8b	0	0	17098

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 185473 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/221ada155c09.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/221ada155c09.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/221ada155c09.tar.gz (if generated)